

A BEGINNER'S FIELD GUIDE

Python for Machine Learning

From raw, messy data to a trained model — the practical toolkit.

You learned the ideas in Phase 1. Now you'll get your hands dirty: load real data, clean it, chart it, and run the full scikit-learn workflow yourself.

Phase 2 • Python for ML

Where Phase 2 fits

Phase 1 gave you the mental model — what ML is, the main algorithms, and how to judge a model. Phase 2 gives you the *hands*: the four Python tools every practitioner uses every day, and the workflow that ties them together.

THE FOUR TOOLS YOU'LL MEET

NumPy — fast number-crunching on arrays. **pandas** — spreadsheets in code: load, filter, group, join. **Matplotlib & Seaborn** — turning data into charts. **scikit-learn** — the train / predict / evaluate engine. Real ML work is roughly 80% data wrangling and 20% modelling, so we spend most of our time on the first three.

INSTALL EVERYTHING (ONE LINE)

Open a terminal and run: `pip install pandas numpy matplotlib seaborn scikit-learn`. That's the entire toolkit for this phase.

Versions used here (all current as of 2026): pandas 3.0, NumPy 2.4, scikit-learn 1.8, seaborn 0.13, Matplotlib 3.10. The code works on any recent version.

What's inside

A **Data Handling** — getting data into shape

NumPy arrays · pandas load/filter/group/merge · nulls & outliers · feature engineering

B **Visualisation** — seeing your data

Matplotlib line/bar/scatter · Seaborn heatmaps & distributions · reading model charts · feature importance

C **Scikit-learn Basics** — the workflow

train/test split · fit→predict→evaluate · pipelines · cross-validation

D **Hands-On Project** — "The Penguin Classifier"

A complete real workflow: clean → chart → pipeline → cross-validate → explain

+ **Glossary** & what comes next

PART A

Data Handling

Models are only as good as the data you feed them. This is where the real work lives.

A.1 NumPy — fast maths on arrays

NumPy gives you the *array*: a grid of numbers that Python can crunch extremely fast. pandas and scikit-learn are both built on top of it, so it helps to recognise it.

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a.mean())          # 25.0 — averages the whole array at once
print(a * 2)              # [20 40 60 80] — maths applies to every element
print(a[a > 20])          # [30 40] — keep only values above 20
```

PLAIN-ENGLISH ANALOGY

A plain Python list is like writing numbers on separate sticky notes — to add them all you handle each note. A NumPy array is a calculator that operates on the whole row at once. Same idea, vastly faster.

A.2 pandas — spreadsheets in code

pandas is the workhorse. Its main object, the **DataFrame**, is a table with named columns — think Excel, but programmable. Here are the four moves you'll use constantly.

Load

```
import pandas as pd
df = pd.read_csv("sales.csv") # read a file into a table
df.head()                    # peek at the first 5 rows
df.info()                    # column names, types, and missing counts
```

Filter (keep the rows you want)

```
big = df[df["amount"] > 100] # only rows where amount > 100
uk = df[df["country"] == "UK"] # only UK rows
```

Group (summarise by category)

```
df.groupby("country")["amount"].mean() # average amount per country
```

This is the single most useful data move there is: "show me the average/total/count *for each* group." It answers most business questions on its own.

Merge (join two tables)

```
full = pd.merge(orders, customers, on="customer_id") # glue tables by a shared key
```

PLAIN-ENGLISH ANALOGY

Merge is like **VLOOKUP in Excel**: you have an orders table and a customers table that share a *customer_id*; merge stitches each order to its customer's details into one combined table.

A.3 Handling nulls and outliers

Real data is messy. Two problems show up almost every time.

Nulls (missing values)

A **null** is an empty cell — a survey someone skipped, a sensor that failed. Most models refuse to run with gaps, so you must deal with them. You have two honest choices:

- **Drop** the rows or columns with gaps — fine when there are few.
- **Fill** them in (called *imputation*) — e.g. replace a missing number with the column's median, or a missing category with the most common value.

```
df.isnull().sum() # count gaps in each column
df = df.dropna(subset=["price"]) # drop rows missing a price
df["age"] = df["age"].fillna(df["age"].median()) # fill with the median
```

Outliers (extreme oddballs)

An **outlier** is a value far outside the normal range — a typo'd price of \$1,000,000, an age of 250. They can drag a model off course. Spot them with a quick summary or a box plot, then decide: fix a clear typo, cap extreme values, or remove genuine errors. Don't blindly delete — sometimes the oddball is the most interesting case (that's literally fraud detection in Phase 3).

WATCH OUT

Always inspect *before* you delete. An "outlier" might be a real, important event. And never fill nulls using information from your test set — that quietly leaks answers and inflates your scores. (Pipelines, in Part C, prevent this automatically.)

A.4 Feature engineering basics

A **feature** is an input clue the model learns from. **Feature engineering** is creating better clues from the raw data — often the highest-impact thing you can do, more than swapping algorithms.

- **Combine columns**: turn *height* and *weight* into a single *BMI*.

- **Pull pieces out of dates:** from a timestamp, extract *day-of-week* or *is_weekend* — often far more predictive than the raw date.
- **Encode categories as numbers:** models need numbers, so "red/green/blue" becomes columns of 0s and 1s (called *one-hot encoding*).
- **Scale numbers** to a common range so a big-numbered column (salary) doesn't drown out a small one (years of experience).

KEY IDEA

Better features usually beat fancier models. Time spent understanding and reshaping your data pays off more than time spent tuning an algorithm.

Visualisation

Before you model, *look*. A chart reveals in seconds what a table of numbers hides.

B.1 Matplotlib — the three charts you'll use most

Matplotlib is the base plotting library. Three chart types cover most needs:

- **Line plot** — how something changes over time (sales per month).
- **Bar plot** — comparing categories (revenue per region).
- **Scatter plot** — the relationship between two numbers (ad spend vs. sales); this is where you spot correlations and outliers.

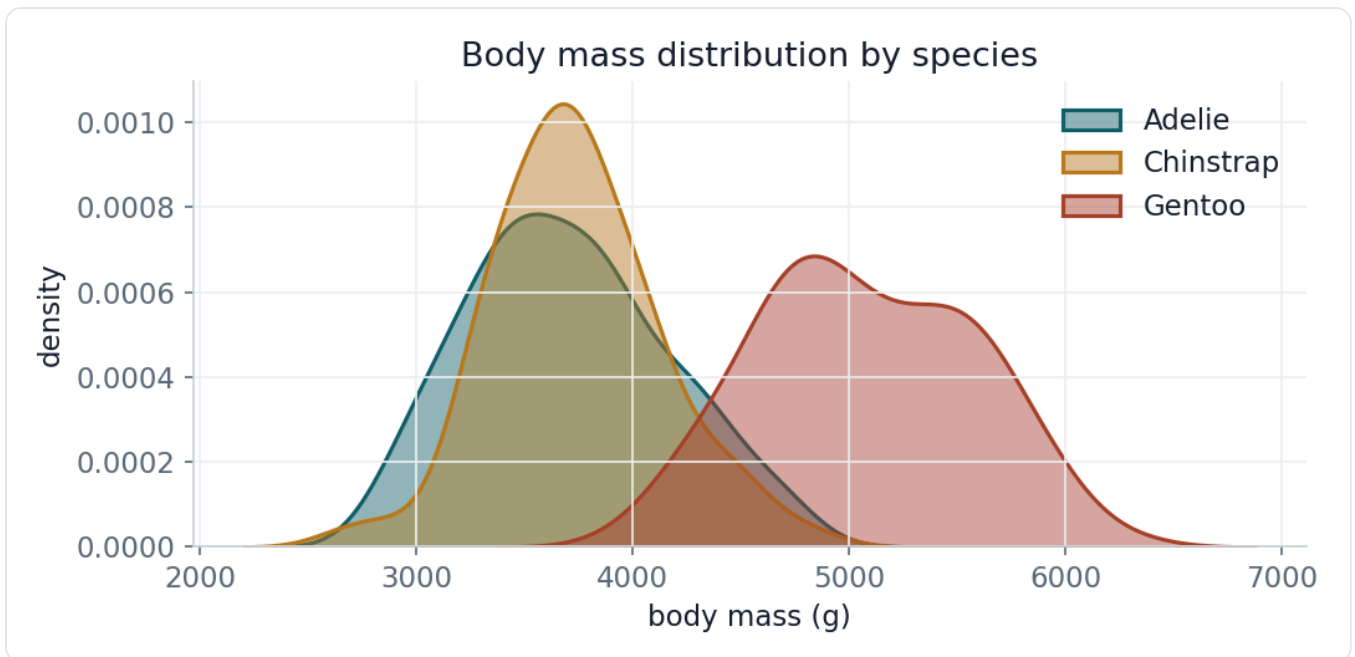
```
import matplotlib.pyplot as plt
plt.plot(months, sales)      # line: a trend over time
plt.bar(regions, revenue)   # bar: compare categories
plt.scatter(ad_spend, sales) # scatter: relationship between two numbers
plt.show()
```

B.2 Seaborn — beautiful charts in one line

Seaborn sits on top of Matplotlib and makes statistical charts effortless. Two are especially useful early on.

Distribution plot — the shape of one column

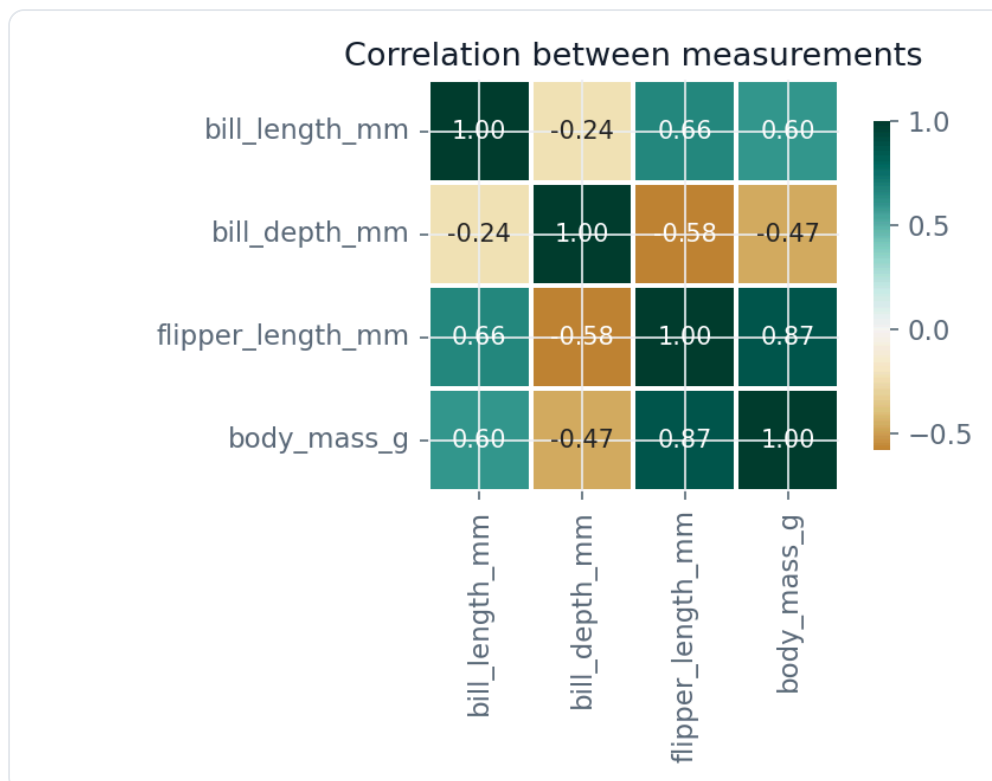
Shows where values cluster. Below, the three penguin species clearly differ in body mass — Gentoos are noticeably heavier. That separation is a hint the data will be easy to classify.



```
sns.kdeplot(data=df, x="body_mass_g", hue="species", fill=True)
```

Correlation heatmap — what moves with what

A grid showing how strongly each pair of numeric columns moves together, from -1 (opposite) through 0 (unrelated) to +1 (in lockstep). Here, flipper length and body mass are strongly linked (0.87) — bigger flippers, heavier bird.



```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="BrBG")
```

KEY IDEA

Correlation shows that two things move together — not that one *causes* the other. Ice-cream sales and sunburns rise together; neither causes the other (the sun does).

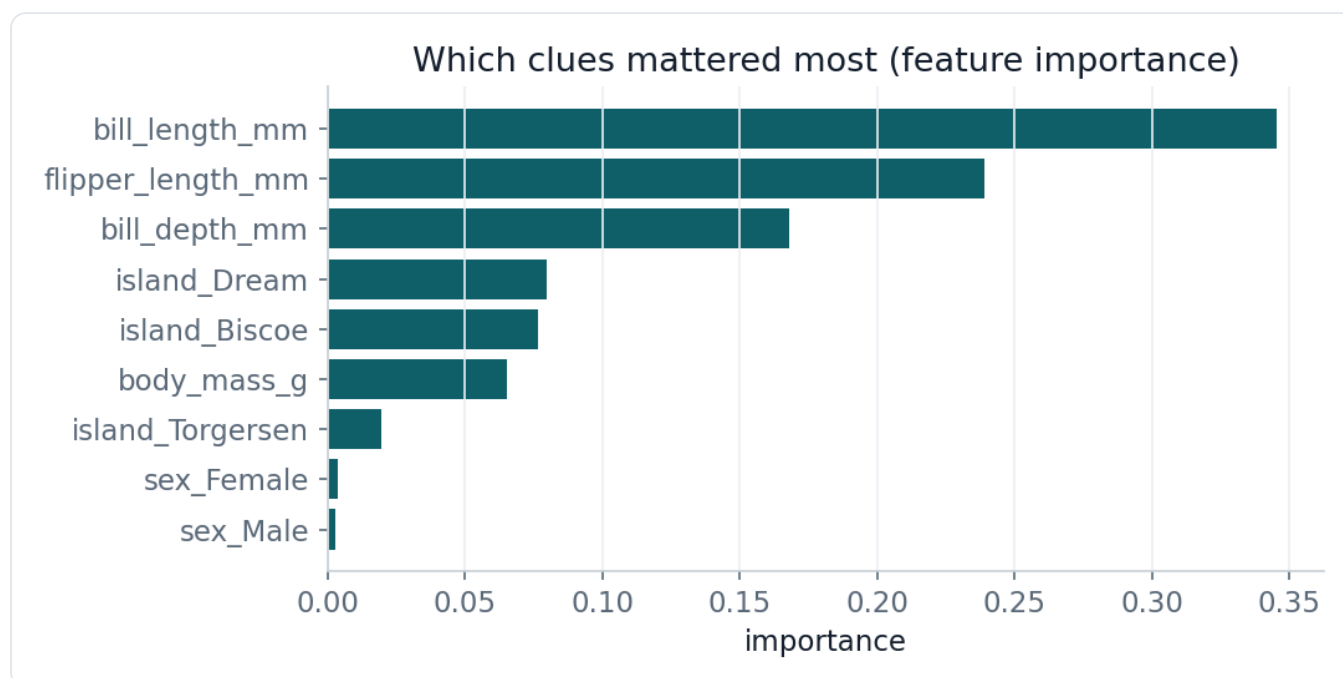
B.3 Reading model output charts

You'll meet a few charts again and again when judging models (all from Phase 1's metrics):

- **Confusion matrix heatmap** — a coloured grid of right vs. wrong calls; you want the diagonal bright and the rest dark.
- **ROC curve** — a curve hugging the top-left corner is a strong model; the area under it is the AUC score.
- **Learning / validation curve** — training and test scores plotted together; a wide gap is the visual signature of overfitting from Phase 1.

B.4 Plotting feature importance

After training, many models can tell you *which clues mattered most*. Plotting this as a bar chart is the fastest way to explain a model to anyone — technical or not. Below, bill length was the most useful clue for telling penguin species apart, while the bird's sex barely mattered.



WHY THIS MATTERS

A stakeholder rarely wants the maths. "The model mostly looks at bill length and flipper length" is a sentence anyone understands — and it builds trust. We'll lean on this heavily in Phase 4.

Scikit-learn Basics

The same four-step rhythm powers almost every model you'll ever build.

C.1 Train/test split — the honesty rule

From Phase 1: never judge a model on data it studied. So before anything else, split your data — typically 80% to learn from, 20% held back to test on.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

`X` = the input features (the clues). `y` = the target (the answer). `random_state=42` just makes the random split repeatable so you get the same result each run.

C.2 The fit → predict → evaluate workflow

Every scikit-learn model speaks the same three verbs. Learn them once and you can use *any* model.

Verb	What it does	Phase 1 name
<code>.fit(...)</code>	Learn the patterns from the training data.	Training
<code>.predict(...)</code>	Make predictions on new data.	Inference
<code>.score(...)</code>	Check how good those predictions are.	Evaluation

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)           # learn
preds = model.predict(X_test)        # predict on unseen data
print(model.score(X_test, y_test))   # evaluate (accuracy here)
```

C.3 Pipelines — bundling steps so nothing leaks

Your data usually needs preparation (fill nulls, scale numbers, encode categories) *before* the model sees it. A **Pipeline** bolts those prep steps and the model into one object. Two big wins:

- **No leakage:** the prep is learned from training data only, then applied to test data — automatically and correctly.
- **One object:** you `.fit()` and `.predict()` the whole thing at once; no chance of forgetting a step in production.

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

pipe = Pipeline([
    ("fill", SimpleImputer(strategy="median")), # 1. fill nulls
    ("scale", StandardScaler()), # 2. scale numbers
    ("model", RandomForestClassifier()) # 3. the model
])
pipe.fit(X_train, y_train) # all three steps run in order, safely

```

PLAIN-ENGLISH ANALOGY

A pipeline is a factory conveyor belt: raw data goes in one end, passes each station (clean → scale → model) in the right order every single time, and a prediction comes out the other end. No skipped steps, no contamination.

C.4 Cross-validation — a fairer test

A single train/test split can be lucky or unlucky depending on which rows landed where. **Cross-validation** removes that luck: it splits the data into, say, 5 parts ("folds"), trains on 4 and tests on the 5th, then rotates so every part gets a turn as the test set. You get 5 scores instead of 1 — their average is far more trustworthy.

```

from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipe, X, y, cv=5) # 5 folds
print(scores.mean(), scores.std()) # average and how much it wobbles

```

KEY IDEA

Report the *average* score and how much it varies. A model scoring 0.91 ± 0.01 is reliable; one scoring 0.91 ± 0.15 is a coin toss dressed up as a number.

The Penguin Classifier

One real workflow that uses every tool in this phase, end to end.

We'll predict a penguin's **species** from its body measurements — loading real, *messy* data, cleaning it, charting it, then running a full scikit-learn pipeline with cross-validation. Every number and chart below is the real output of this exact code.

0 Setup

```
pip install pandas seaborn scikit-learn matplotlib
```

Save the steps below into `penguins.py` and run with `python penguins.py`. The dataset comes built into seaborn — nothing to download.

1 Load and inspect

```
import seaborn as sns, pandas as pd
df = sns.load_dataset("penguins")
print(df.shape)
print(df.isnull().sum()) # where are the gaps?
print(df.groupby("species")["body_mass_g"].mean()) # a groupby summary
```

REAL OUTPUT

```
(344, 7) <- 344 penguins, 7 columns

missing values per column:
  bill_length_mm      2
  bill_depth_mm      2
  flipper_length_mm  2
  body_mass_g        2
  sex              11 <- 11 penguins have no recorded sex

average body mass by species (g):
  Adelie      3701
  Chinstrap  3733
  Gentoo     5076 <- Gentoo are clearly the heavy ones
```

2 Clean the gaps

```
clean = df.dropna(subset=["bill_length_mm", "flipper_length_mm",
                          "bill_depth_mm", "body_mass_g"])
```

```
clean["sex"] = clean["sex"].fillna(clean["sex"].mode()[0]) # fill sex with most common
```

REAL OUTPUT

```
rows after cleaning: 342      nulls left: 0
```

3 Look before you model

A distribution plot and a correlation heatmap (the exact charts from Part B) confirm the species separate well and the measurements are related — good signs before we train.

4 Build the pipeline and split

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score

X = clean.drop(columns=["species"]); y = clean["species"]
num = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
cat = ["island", "sex"]

prep = ColumnTransformer([
    ("num", Pipeline([("fill", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])

pipe = Pipeline([("prep", prep),
                  ("model", RandomForestClassifier(n_estimators=200, random_state=42))])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          random_state=42, stratify=y)

pipe.fit(X_tr, y_tr)
print("test accuracy:", pipe.score(X_te, y_te))
```

REAL OUTPUT

```
test accuracy: 1.0
```

5 Cross-validate for a trustworthy score

```
cv = cross_val_score(pipe, X, y, cv=5)
print(cv)
print("mean:", cv.mean().round(3), "wobble:", cv.std().round(3))
```

REAL OUTPUT

```
[1.000, 0.986, 0.985, 0.985, 1.000]
mean: 0.991  wobble: 0.007
```

A mean of **0.991 with a tiny ± 0.007 wobble** means the model is genuinely reliable, not lucky on one split. This is a far stronger claim than the single **1.0** from step 4.

6 Explain what it learned

Plotting feature importance (the Part B chart) shows the model leaned mostly on **bill length** and **flipper length** — exactly the measurements a biologist would use. When a model's logic matches real-world sense, trust goes up.

STRETCH CHALLENGE

1. Swap `RandomForestClassifier` for `LogisticRegression` — does cross-validation still score as high? 2. Predict `sex` instead of `species` (much harder — watch the score drop and feel why some problems are tougher).
3. Print a confusion matrix from Phase 1 on the test set.

PHASE 2 CHECKLIST — YOU CAN NOW...

✓ load & inspect data with pandas • ✓ filter, group and merge tables • ✓ handle nulls and outliers • ✓ engineer and encode features • ✓ chart distributions, correlations and feature importance • ✓ build a leak-proof pipeline • ✓ run cross-validation and report a trustworthy score.

Plain-English glossary (Phase 2)

Term	In plain words
DataFrame	A table of data in pandas — like a programmable spreadsheet.
Array	A fast grid of numbers (NumPy); the raw material under pandas and sklearn.
Null / NaN	A missing/empty value that must be dropped or filled.
Imputation	Filling in missing values (e.g. with the median).
Outlier	An extreme value far outside the normal range.
Feature engineering	Creating better input clues from raw data.
One-hot encoding	Turning a category column into 0/1 columns models can read.
Scaling	Putting numbers on a comparable range so big numbers don't dominate.
Correlation	How strongly two numbers move together (−1 to +1).
Pipeline	Prep steps + model bundled into one safe, repeatable object.
Cross-validation	Testing on several rotating splits for a fairer score.
Data leakage	Letting test-set info sneak into training, faking good scores.

What's next — Phase 3

Applied ML: Real Use Cases. With the toolkit in hand, Phase 3 puts it to work on problems you'll actually be asked to solve: **anomaly & fraud detection** (Isolation Forest, log analysis), **forecasting** (time series, ARIMA, Prophet), **classification** for triage and risk scoring, **text & NLP** (sentiment, document triage, LLM APIs), plus an intro to **MLOps** — versioning, retraining, drift monitoring, and serving models via APIs — and **interpretability** with SHAP. Each with its own hands-on project.

Machine Learning, From Zero · Phase 2 of 4 · Library behaviour follows the official pandas 3.0, scikit-learn 1.8 and seaborn 0.13 documentation. Charts and outputs were generated by running the code shown. This guide is for learning and reproduces no copyrighted text.